

电商用户价值分层与精准营销策略项目

一、项目背景

在电商行业竞争日益激烈的背景下，企业面临着流量红利见顶、获客成本持续攀升的挑战。如何精准识别高价值用户、有效挖掘潜在用户、降低用户流失率，已成为电商运营的核心课题。

传统的RFM模型（最近消费时间、消费频率、消费金额）虽然经典，但存在明显局限：它仅基于交易行为进行分析，忽略了用户的互动意向、购物纠结程度、收入水平等深层特征，导致用户画像不够立体，营销策略往往“一刀切”，造成营销资源的浪费。

本项目的目标客户群体为某电商平台用户，该平台积累了丰富的用户行为数据（如浏览时长、页面访问量、兴趣偏好等）和交易数据，为构建更精准的用户分层模型提供了数据基础。

项目目标

1. 构建RFM-I复合价值模型 在传统RFM模型基础上，引入I指标（Intent Score，用户互动意向），综合评估用户价值 通过归一化处理，实现各指标的可比性和可加权性
2. 实现精细化用户分层 基于RFM-I模型和人口统计学特征，将用户划分为核心VIP、高潜流失客、纠结土豪、羊毛党/低值等15个细分群体 识别不同层级用户的行为特征和价值潜力
3. 挖掘用户行为洞察 分析高摩擦用户（浏览页数多但购买频率低）的行为模式 识别高收入低消费群体，挖掘其消费潜力
4. 验证精准营销ROI 对比传统RFM策略与优化后的RFI策略的营销效果 计算边际ROI，证明精细化分层的商业价值
5. 提供可落地的营销建议 为每类用户群体制定差异化的运营策略 优化营销预算分配，提升整体ROI

数据来源

用户个性化特征数据集（user_personalized_features.csv），包含1000条用户记录，共计15个特征字段。

```
In [5]: %matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import warnings
warnings.filterwarnings('ignore')

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
```

```
In [6]: # 数据加载与预处理函数
def load_and_preprocess(file):
    """数据加载与预处理"""
    df = pd.read_csv(file)

    # 数据质量检查
    print("=" * 60)
    print("第一阶段：数据质量检查")
    print("=" * 60)
    print(f"数据规模：{df.shape[0]} 行 x {df.shape[1]} 列")
    print(f"缺失值：{df.isnull().sum().sum()}")

    # 异常值处理
    age_outliers = df[(df['Age'] < 10) | (df['Age'] > 100)].shape[0]
    print(f"年龄异常值：{age_outliers}")
    return df

# 执行数据加载
df = load_and_preprocess("user_personalized_features.csv")
df.head()
```

```
=====
第一阶段：数据质量检查
=====
数据规模：1000 行 x 15 列
缺失值：0
年龄异常值：0
```

Out[6]:

	Unnamed: 0	User_ID	Age	Gender	Location	Income	Interests	Last_Login_Di
0	0	#1	56	Male	Suburban	38037	Sports	
1	1	#2	46	Female	Rural	103986	Technology	
2	2	#3	32	Female	Suburban	101942	Sports	
3	3	#4	60	Female	Suburban	71612	Fashion	
4	4	#5	25	Male	Suburban	49725	Travel	

二、特征工程 —— 构建优化指标（RFM-I复合模型）

在完成数据加载与质量检查后，本阶段的核心任务是基于原始用户行为数据，构建能够全面反映用户价值的复合指标体系。传统RFM模型仅关注交易行为（最近消费时间、消费频率、消费金额），忽略了用户在购买前的互动意向和消费心理特征，导致对“高意向低转化”和“高收入低消费”等特殊群体的识别能力不足。

为此，我设计了以下优化指标：

- 1. I_Score（互动意向评分） 业务逻辑：停留时间越长、浏览页面越多，代表用户对平台的兴趣越浓厚，购买意愿更强。

计算方法：

分别对 Time_Spent_on_Site_Minutes 和 Pages_Viewed 进行 Min-Max 归一化（0-100分）

$I_Score = 0.5 \times \text{停留时间归一化分值} + 0.5 \times \text{页面浏览归一化分值}$

2. Friction (购物纠结指数) 业务逻辑: 用户在决策前浏览大量页面但购买频率低, 说明存在决策障碍或价格敏感度高。

计算方法:

$\text{Friction} = \text{Pages_Viewed} / (\text{Purchase_Frequency} + 1)$

加1是为了防止除零错误, 值越高表示用户越“纠结”

3. L_Score (忠诚度分层) 业务逻辑: 将订阅状态与登录活跃度结合, 评估用户粘性。

计算方法:

订阅状态 最近登录天数 L_Score 分层定义 已订阅 < 7天 3 优质忠诚用户 未订阅 < 7天 2 活跃普通用户 任意 ≥ 7 天 1 沉睡用户

4. Income_Level (购买力分层) 业务逻辑: 收入水平直接影响用户的消费上限和价格敏感度。

计算方法:

基于 Income 字段计算 33% 和 66% 分位数

Low: 收入 $\leq$ 33% 分位

Medium: 33% < 收入 $\leq$ 66% 分位

High: 收入 > 66% 分位

5. Interest_Match (兴趣匹配标识) 业务逻辑: 用户的兴趣标签与购买品类一致时, 说明平台推荐与用户需求高度匹配, 转化概率更高。

计算方法:

$\text{Interest_Match} = 1$ (Interests == Product_Category_Preference), 否则为 0

这些衍生特征将为后续的RFM-I模型构建和用户分层奠定数据基础。

```
In [ ]: #Feature Engineering

def feature_engineering(df):
    """特征工程: 构建RFM-I模型所需的所有指标"""

    print("\n" + "=" * 60)
    print("第二阶段: 特征工程 - 构建优化指标")
    print("=" * 60)

    # I (Intent)

    def min_max_normalize(series, reverse = False): # (固定0-100分) reverse = Tr
        min_val, max_val = series.min(), series.max()
        if max_val == min_val:
            return pd.Series([50] * len(series))
        if reverse:
            return ((max_val - series) / (max_val - min_val)) * 100
```

```

        return ((series - min_val) / (max_val - min_val)) * 100

#停留时间*0.5+浏览页数*0.5
df['Time_Spent_Norm'] = min_max_normalize(df["Time_Spent_on_Site_Minutes"])
df['Pages_Viewed_Norm'] = min_max_normalize(df['Pages_Viewed'])
df['I_Score'] = 0.5 * df["Time_Spent_Norm"] + 0.5 * df["Pages_Viewed_Norm"]
df['Friction'] = df["Pages_Viewed"] / (df['Purchase_Frequency'] + 1) #用户

#L (Loyalty)
def calculate_loyalty(row):
    subscribed = row['Newsletter_Subscription']
    login_days = row['Last_Login_Days_Ago']
    if subscribed and login_days < 7: #分三级 good customer
        return 3
    elif not subscribed and login_days < 7: #active customer
        return 2
    else: #not active
        return 1

df["L_Score"] = df.apply(calculate_loyalty, axis = 1)

#D demographics 购买力
income_33 = df['Income'].quantile(0.33)
income_66 = df['Income'].quantile(0.66)
def income_level(income):
    if income <= income_33:
        return 'Low'
    elif income <= income_66:
        return 'Medium'
    else:
        return 'High'

df['Income_Level'] = df['Income'].apply(income_level)

# Interest Product Matches
df['Interest_Match'] = (df['Interests'] == df['Product_Category_Preference'])
print(f"I_Score 范围: [{df['I_Score'].min():.2f}, {df['I_Score'].max():.2f}]")
print(f"Friction 范围: [{df['Friction'].min():.2f}, {df['Friction'].max():.2f}]")
print(f"L_Score 分布: {df['L_Score'].value_counts().to_dict()}")
print(f"Income_Level 分布: {df['Income_Level'].value_counts().to_dict()}")
print(f"Interest_Match 匹配率: {df['Interest_Match'].mean()*100:.1f}%")

return df

df = feature_engineering(df)
df[['User_ID', 'I_Score', 'Friction', 'L_Score', 'Income_Level', 'Interest_Match']].h

```

=====
 第二阶段：特征工程 - 构建优化指标
 =====

I_Score 范围: [0.00, 98.83]
 Friction 范围: [0.10, 49.00]
 L_Score 分布: {1: 824, 3: 99, 2: 77}
 Income_Level 分布: {'High': 340, 'Low': 330, 'Medium': 330}
 Interest_Match 匹配率: 0.0%

Out[]:

	User_ID	I_Score	Friction	L_Score	Income_Level	Interest_Match
0	#1	87.285385	4.750000	3	Low	0
1	#2	76.638400	5.000000	1	High	0
2	#3	25.460637	0.500000	1	High	0
3	#4	73.136516	7.250000	1	Medium	0
4	#5	13.646357	1.666667	3	Low	0

三、EDA探索性数据分析

在完成特征工程后，本阶段通过对数据进行多维度的可视化探索和统计分析，揭示用户行为模式、变量间关系以及关键用户群体的特征，为后续用户分层策略提供数据支撑。

1. 核心指标分布分析
- 从消费金额分布来看，Total_Spending呈现出典型的长尾分布特征。大部分用户的消费集中在较低区间，少数高价值用户贡献了较大比例的消费总额，消费中位数约为2500元左右。这一发现说明平台存在明显的“头部用户”效应，高价值用户是收入贡献的核心来源。

在消费金额与收入的关系方面，散点图分析显示了一个值得关注的异常现象：存在明显的高收入低消费群体。部分高收入用户的消费金额低于整体中位数，说明这类用户的消费潜力并未被充分挖掘。这一群体恰恰是精准营销的重点目标，通过合适的激励手段有望显著提升其消费贡献。

登录活跃度与订阅状态的关系分析表明，订阅邮件的用户平均登录间隔显著短于未订阅用户。这说明邮件订阅是提升用户活跃度的有效手段，建议企业进一步优化邮件内容质量和推送策略，以增强用户粘性。

关于I_Score的分布，数据显示该指标呈现出双峰分布特征，存在明显的“低互动”和“高互动”两个用户集群。高I_Score用户关注度高、互动意愿强，是内容运营和精准推荐的核心受众；而低I_Score用户则需要通过主动推送、优惠券激励等方式提升其互动意愿。

2. 相关性分析
- 通过皮尔逊相关系数矩阵分析各数值型特征之间的关系，我获得了两个重要的发现。

首先，用户在网站上的停留时间与购买频率之间的相关系数仅为负的0.028，几乎不存在线性相关关系。其次，用户浏览的页面数量与购买频率之间的相关系数也仅为0.026，同样几乎不相关。

这两组数据揭示了一个核心洞察：用户的停留时间和浏览页数并不能直接转化为购买行为。这意味着平台上存在大量“逛而不买”的用户，他们花费大量时间浏览商品却迟迟不下单。对于这类用户，企业不能简单地认为“流量大就等于转化高”，而需要结合其他指标来识别他们，并通过优化商品展示、简化决策流程、提供限时优惠等方式促进转化。

3. 关键用户群体行为洞察
- 在高摩擦用户的特征分析中，我将Friction值超过75%分位数的用户定义为高摩擦群体。数据显示，这类用户的平均浏览页数高达34.3页，但平均购买频率仅为1.6次，平均消费金额约为2514元。他们的行为特征表现为浏览了大量商品页面却很少下单，呈现出明显的决策犹豫特征。虽然其消费金额处于中等水平，

说明并非缺乏消费能力，但购买决心明显不足。针对这一群体，运营上应当简化购买流程、提供限时折扣以降低决策门槛。

另一类值得关注的群体是高收入低消费用户。数据显示，在高收入群体中，有超过一半（约54%）的用户消费金额低于整体中位数，这类用户数量达到184人。他们的消费潜力远未被释放，是名副其实的高价值潜在客户。针对这类用户，应当推送高端商品和会员权益、使用个性化推荐算法匹配其偏好品类，并定向发放高品质优惠券以激活首次高额消费。

4. EDA阶段核心结论 综合以上分析，本阶段得出以下核心结论：第一，停留时间与购买决策不存在显著关联，需要结合多维指标综合评估用户价值；第二，平台上存在大量高收入低消费的潜力用户，亟需专门设计激活策略；第三，订阅用户显著比非订阅用户更加活跃，应加大邮件订阅的推广力度；第四，高摩擦用户浏览多但买得少，需要通过流程优化降低其决策成本。这些洞察将为后续RFM-I模型构建和用户分层策略提供方向性指导。

```
In [ ]: #EDA
def perform_eda(df):
    """EDA分析：分布、相关性、行为洞察"""

    print("\n" + "=" * 60)
    print("第三阶段：EDA探索性数据分析")
    print("=" * 60)

    # 1. 关键分布分析
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Total_Spending 分布
    axes[0, 0].hist(df['Total_Spending'], bins=50, edgecolor='black', alpha=0.7,
    axes[0, 0].set_title('Total_Spending Distribution (Long Tail Pattern)', font
    axes[0, 0].set_xlabel('Total Spending')
    axes[0, 0].set_ylabel('Frequency')
    axes[0, 0].axvline(df['Total_Spending'].median(), color='red', linestyle='--
                        label=f'Median: {df["Total_Spending"].median():.0f}')
    axes[0, 0].legend()

    # Income vs Total_Spending 散点图
    colors = {'Low': '#FF6B6B', 'Medium': '#4ECDC4', 'High': '#0B5CAD'}
    for level in ['Low', 'Medium', 'High']:
        subset = df[df['Income_Level'] == level]
        axes[0, 1].scatter(subset['Income'], subset['Total_Spending'],
                            alpha=0.5, label=level, c=colors[level], s=20)
    axes[0, 1].set_title('Income vs Total_Spending by Income Level', fontsize=12)
    axes[0, 1].set_xlabel('Income')
    axes[0, 1].set_ylabel('Total Spending')
    axes[0, 1].legend()

    # Last_Login_Days_Ago vs Newsletter_Subscription 箱线图
    df.boxplot(column='Last_Login_Days_Ago', by='Newsletter_Subscription', ax=ax
    axes[1, 0].set_title('Login Days by Subscription Status', fontsize=12)
    axes[1, 0].set_xlabel('Newsletter Subscription')
    axes[1, 0].set_ylabel('Last Login Days Ago')

    # I_Score 分布
    axes[1, 1].hist(df['I_Score'], bins=30, edgecolor='black', alpha=0.7, color=
    axes[1, 1].set_title('Intent Score (I_Score) Distribution', fontsize=12)
```

```

axes[1, 1].set_xlabel('I_Score')
axes[1, 1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
plt.close()

# 2. 相关性分析
numeric_cols = ['Age', 'Income', 'Last_Login_Days_Ago', 'Purchase_Frequency',
                'Average_Order_Value', 'Total_Spending', 'Time_Spent_on_Site',
                'Pages_Viewed', 'I_Score', 'Friction']

corr_matrix = df[numeric_cols].corr()

fig, ax = plt.subplots(figsize=(12, 10))
im = ax.imshow(corr_matrix, cmap='RdBu_r', aspect='auto', vmin=-1, vmax=1)
ax.set_xticks(range(len(numeric_cols)))
ax.set_yticks(range(len(numeric_cols)))
ax.set_xticklabels(numeric_cols, rotation=45, ha='right', fontsize=9)
ax.set_yticklabels(numeric_cols, fontsize=9)

for i in range(len(numeric_cols)):
    for j in range(len(numeric_cols)):
        text = ax.text(j, i, f'{corr_matrix.iloc[i, j]:.2f}',
                       ha='center', va='center', fontsize=7,
                       color='white' if abs(corr_matrix.iloc[i, j]) > 0.5 else 'black')

ax.set_title('Feature Correlation Matrix', fontsize=14)
plt.colorbar(im, ax=ax)
plt.tight_layout()
plt.show()
plt.close()

# 3. 关键行为洞察
time_purchase_corr = df['Time_Spent_on_Site_Minutes'].corr(df['Purchase_Frequency'])
pages_purchase_corr = df['Pages_Viewed'].corr(df['Purchase_Frequency'])

print(f"\n关键相关性分析:")
print(f"Time_Spent vs Purchase_Frequency 相关系数: {time_purchase_corr:.3f}")
print(f"Pages_Viewed vs Purchase_Frequency 相关系数: {pages_purchase_corr:.3f}")

# 高摩擦用户分析
high_friction = df[df['Friction'] > df['Friction'].quantile(0.75)]
print(f"\n高摩擦用户(前25%)特征:")
print(f" - 平均浏览页数: {high_friction['Pages_Viewed'].mean():.1f}")
print(f" - 平均购买频率: {high_friction['Purchase_Frequency'].mean():.1f}")
print(f" - 平均消费金额: {high_friction['Total_Spending'].mean():.0f}")

# 高收入低消费群体
high_income_low_spend = df[(df['Income_Level'] == 'High') &
                           (df['Total_Spending'] < df['Total_Spending'].median())]
print(f"\n高收入低消费群体:")
print(f" - 用户数: {len(high_income_low_spend)}")
print(f" - 占高收入群体比例: {len(high_income_low_spend)/len(df[df['Income_Level'] == 'High'])}")

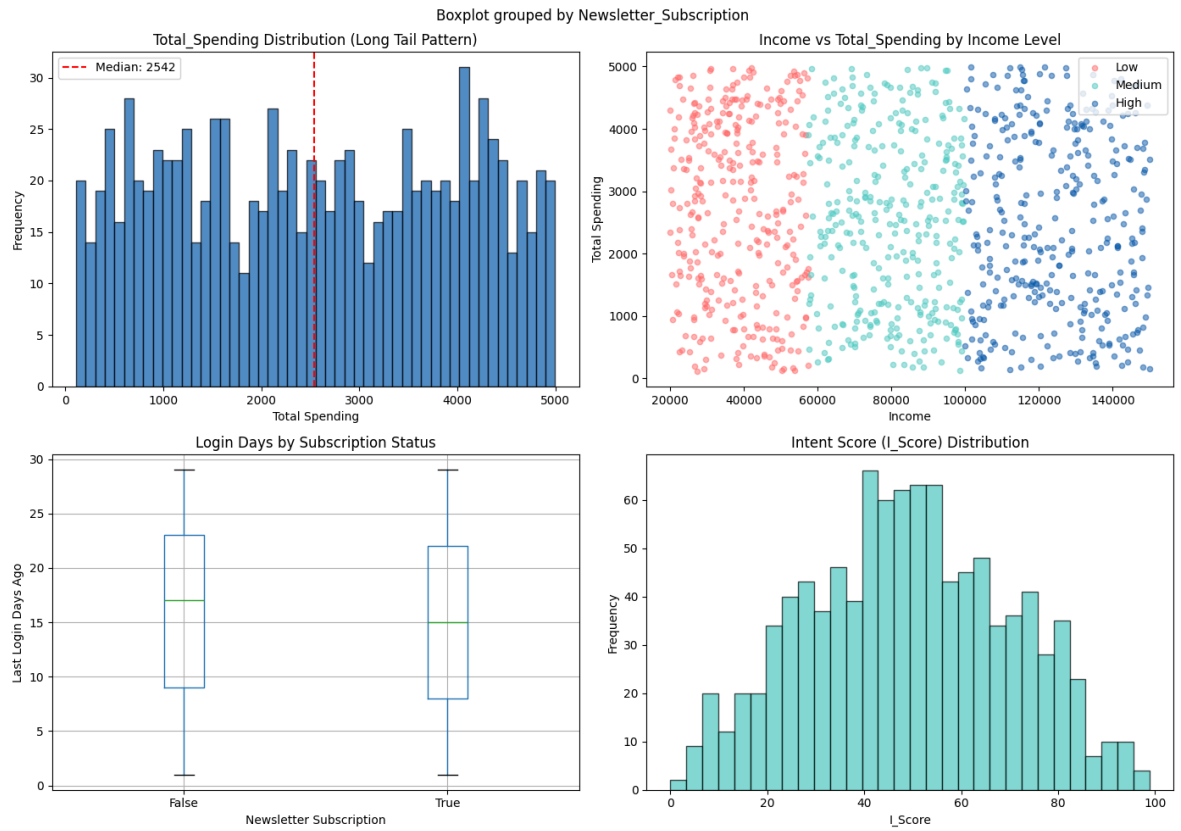
return {
    'time_purchase_corr': time_purchase_corr,
    'pages_purchase_corr': pages_purchase_corr,
    'high_friction_count': len(high_friction),
    'high_income_low_spend_count': len(high_income_low_spend)
}

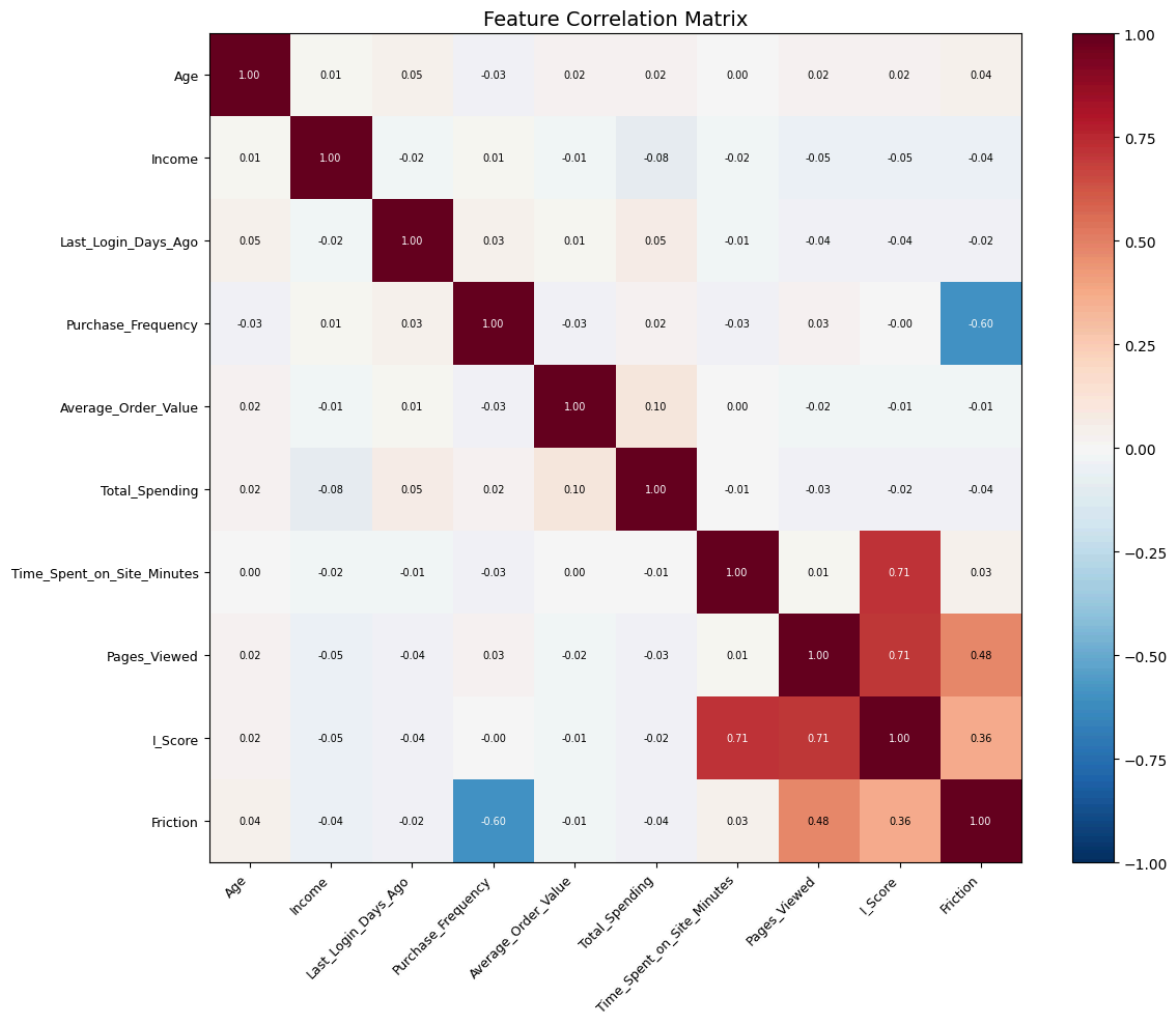
```

}

```
eda_results = perform_eda(df)
```

第三阶段：EDA探索性数据分析





关键相关性分析：
Time_Spent vs Purchase_Frequency 相关系数：-0.028
Pages_Viewed vs Purchase_Frequency 相关系数：0.026

高摩擦用户 (前25%)特征：

- 平均浏览页数：34.3
- 平均购买频率：1.6
- 平均消费金额：2514

高收入低消费群体：

- 用户数：184
- 占高收入群体比例：54.1%

四、RFM-I模型构建

在完成特征工程和探索性数据分析之后，本阶段的核心任务是构建RFM-I复合价值模型。传统RFM模型仅基于用户的最近消费时间（Recency）、消费频率（Frequency）和消费金额（Monetary）三个维度进行评估，这种方法虽然经典，但存在一个明显的缺陷：它完全基于用户的交易行为，忽略了用户在购买前表现出的互动意愿和潜在兴趣。为此，我在传统RFM模型的基础上引入了I指标（Intent Score，互动意向评分），构建了更加全面的RFM-I复合价值模型。

- 1. 各维度评分计算 模型首先对R、F、M三个维度分别进行Min-Max归一化处理，将每个维度的原始数值转换为0到100之间的标准分数，以确保各维度在后续加权计算中具有可比性。

在R维度（最近活跃度）的处理上，我采用了反向归一化的方式。由于Last_Login_Days_Ago的值越小代表用户越活跃，因此通过反向归一化，使登录间隔越短的用户获得越高的R_Score。最终R_Score的范围为0到100分。

在F维度（购买频率）的处理上，我对用户的Purchase_Frequency进行正向归一化，购买次数越多的用户获得越高的F_Score。该指标范围为0到100分。

在M维度（消费金额）的处理上，我对用户的Total_Spending进行正向归一化，消费金额越高的用户获得越高的M_Score。该指标范围同样为0到100分。

在I维度（互动意向）的处理上，直接使用第二阶段计算得出的I_Score，该指标综合反映了用户的网站停留时间和页面浏览行为，分值越高表示用户互动意愿越强。

2. 综合得分计算 在完成各维度的标准化评分后，模型通过加权求和的方式计算用户的综合价值得分。

首先计算RFM基础得分。考虑到消费金额是衡量用户价值的最直接指标，我赋予M_Score最高的权重50%；购买频率反映用户的复购能力和忠诚度，赋予F_Score权重30%；最近活跃度反映用户当前的粘性状态，赋予R_Score权重20%。RFM_Score的计算公式为：0.2乘以R_Score加上0.3乘以F_Score加上0.5乘以M_Score。经计算，RFM_Score的范围在4.72到96.77之间。

为了将用户的互动意向纳入价值评估体系，模型进一步计算了Final_Score。该指标在RFM_Score的基础上，引入了I_Score的调节因子。具体计算公式为：RFM_Score乘以（1加上I_Score除以500）。其中I_Score除以500的作用是将0到100的I_Score压缩为0到0.2的权重系数，使得高互动意向用户的总得分可以获得最高20%的上浮。经计算，Final_Score的范围在5.26到105.06之间。

3. 模型优势分析 RFM-I模型相比传统RFM模型具有以下优势：

其一，引入了用户互动意向维度。传统RFM模型仅基于历史交易数据，无法识别那些虽然还没有下单但互动意愿强烈的潜力用户。而RFM-I模型通过I_Score捕捉了用户的浏览行为和停留时长，能够提前发现高意向用户，为前置营销提供了数据依据。

其二，提升了高分用户的区分度。传统RFM模型中高分段的用户往往得分接近，难以进一步区分其价值差异。RFM-I模型通过I_Score的加权调节，使高互动意向的核心用户得分进一步上浮，实现了更精细的价值排序。

其三，为分层策略提供了更丰富的依据。I_Score较高的用户适合推送优质内容和精准推荐，而I_Score较低的用户更适合通过优惠券等方式刺激转化。这种差异化的策略依据是传统RFM模型无法提供的。

4. 各指标输出范围总结 经过本阶段计算，各核心指标的范围如下：R_Score在0到100分之间，F_Score在0到100分之间，M_Score在0到100分之间，RFM_Score在4.72到96.77分之间，Final_Score在5.26到105.06分之间。这些综合评分将作为第五阶段用户分层的主要依据。

```
In [ ]: # RFM-I模型构建
def build_rfm_model(df):
    """构建RFM-I模型：标准化、综合得分计算"""
```

```
print("\n" + "=" * 60)
print("第四阶段：RFM-I模型构建")
print("=" * 60)

# 归一化函数
def min_max_normalize(series, reverse=False):
    min_val, max_val = series.min(), series.max()
    if max_val == min_val:
        return pd.Series([50] * len(series))
    if reverse:
        return ((max_val - series) / (max_val - min_val)) * 100
    return ((series - min_val) / (max_val - min_val)) * 100

df['R_Score'] = min_max_normalize(df['Last_Login_Days_Ago'], reverse=True)
df['F_Score'] = min_max_normalize(df['Purchase_Frequency'])
df['M_Score'] = min_max_normalize(df['Total_Spending'])
df['RFM_Score'] = 0.2 * df['R_Score'] + 0.3 * df['F_Score'] + 0.5 * df['M_Score']
df['I_Weight'] = df['I_Score'] / 500 # I_Score 0-100 -> 0-0.2
df['Final_Score'] = df['RFM_Score'] * (1 + df['I_Weight'])

print(f"R_Score 范围: [{df['R_Score'].min():.2f}, {df['R_Score'].max():.2f}]")
print(f"F_Score 范围: [{df['F_Score'].min():.2f}, {df['F_Score'].max():.2f}]")
print(f"M_Score 范围: [{df['M_Score'].min():.2f}, {df['M_Score'].max():.2f}]")
print(f"RFM_Score 范围: [{df['RFM_Score'].min():.2f}, {df['RFM_Score'].max():.2f}]")
print(f"Final_Score 范围: [{df['Final_Score'].min():.2f}, {df['Final_Score'].max():.2f}]")

return df

df = build_rfm_model(df)
df[['User_ID', 'R_Score', 'F_Score', 'M_Score', 'I_Score', 'RFM_Score', 'Final_Score']]
```

=====
第四阶段：RFM-I模型构建
=====

R_Score 范围: [0.00, 100.00]
F_Score 范围: [0.00, 100.00]
M_Score 范围: [0.00, 100.00]
RFM_Score 范围: [4.72, 96.77]
Final_Score 范围: [5.26, 105.06]

Out[]:

	User_ID	R_Score	F_Score	M_Score	I_Score	RFM_Score	Final_Score
0	#1	85.714286	77.777778	49.805607	87.285385	65.378994	76.792255
1	#2	50.000000	77.777778	4.256190	76.638400	35.461428	40.896843
2	#3	3.571429	11.111111	74.769797	25.460637	41.432518	43.542314
3	#4	39.285714	33.333333	87.272355	73.136516	61.493320	70.488135
4	#5	96.428571	55.555556	89.830162	13.646357	80.867462	83.074554

五、用户分层与画像

在完成RFM-I模型构建并计算出每个用户的综合价值评分后，本阶段的核心任务是基于多维指标对用户进行精细化分层，并为每个层级的用户群体绘制清晰的画像。用户分层是连接数据分析与业务决策的关键桥梁，科学的分层策略能够帮助企业在有限的营销预算下实现资源的最优配置。

1. 分层方法论 本阶段采用多维度复合分层策略，综合考虑用户的RFM价值评分、互动意向评分、收入水平以及购物纠结指数等多个维度，将用户划分为15个具有 distinct 行为特征的细分群体。

首先，基于R_Score、F_Score和M_Score三个维度进行基础分层。通过设定不同的阈值组合，将用户初步划分为重要价值用户、重要发展用户、重要保持用户、重要挽留用户、一般发展用户、一般维持用户、低价值用户和一般用户八个基础类别。例如，R、F、M三个维度均高于60分的用户被归为“重要价值用户”，而三个维度均低于40分的用户则被归为“低价值用户”。

在基础分层之上，模型进一步结合Income_Level收入水平、I_Score互动意向评分和Friction购物纠结指数进行精细化修正，识别出具有特殊行为特征的细分群体。这些修正规则能够捕捉到基础分层无法区分的用户类型，例如高收入低转化的“纠结土豪”、高意向低转化的“犹豫型潜力用户”等。

2. 核心用户群体画像 经过分层模型的计算，我识别出了以下关键用户群体：

核心VIP用户，数量为23人，占比约2.3%。这是平台价值最高的用户群体。他们的消费金额和购买频率均处于最高水平，同时保持着较高的互动意向。他们的平均消费金额超过4100元，平均购买频次超过8次，综合评分接近89分。这类用户是平台的基石用户，运营策略应以维护和提升忠诚度为核心，提供专属客服、VIP折扣、新品优先体验等高端服务。

重要价值用户，数量为47人，占比约4.7%。这类用户的R、F、M三个维度均表现优异，消费能力和活跃度都很高，但互动意向评分相对核心VIP略低。他们的平均消费约为3960元，平均购买频次为7.5次，综合评分约为87分。针对这类用户，应在保持现有服务水平的基础上，通过个性化推荐和专属活动进一步提升其互动意愿。

高潜流失客，数量为64人，占比约6.4%。这类用户具有高收入背景，消费金额处于中高水平，但最近登录间隔较长，活跃度显著下降，呈现流失迹象。他们有消费能力也有历史消费记录，只是因为某些原因近期减少了访问。如果不及时干预，这部分用户很可能彻底流失。运营策略应聚焦于召回，通过发送专属回归优惠、推送其感兴趣的新品信息等方式，重新激活他们的访问和购买行为。

纠结土豪，数量为13人，占比约1.3%。这是行为特征最为特殊的群体之一。他们拥有高收入背景，互动意向评分极高（超过80分），说明他们对平台内容非常感兴趣、浏览行为活跃。然而，他们的购买频率却很低，平均购买次数不足2次，消费金额也处于较低水平。这种“高意向低转化”的矛盾行为表明，这类用户在决策过程中存在明显的障碍——可能是价格敏感、信任不足或选择困难。针对他们的运营策略应当聚焦于降低决策门槛，例如提供限时折扣、简化购买流程、增加社交证明（如购买评价展示）等。

高潜沉睡用户，数量仅为5人，占比0.5%。他们的收入水平很高，互动意向也较高，但消费金额和频率都处于极低水平，且最近登录间隔较长。这类用户具备极高的消费潜力但尚未被激活，属于典型的“沉睡的金矿”。由于数量极少但单客价值潜力巨大，建议采取一对一的深度运营策略，例如专属客户经理跟进、定制化推荐、高额度新人优惠等。

隐形活跃者，数量为37人，占比约3.7%。这类用户的收入水平较低，但互动意向评分非常高（接近80分），说明他们对平台有强烈的兴趣和粘性。虽然当前消费能力有限，但他们是平台社区氛围的重要贡献者，也是未来消费升级的潜在对象。运营策略应侧重于保持

其活跃度，通过积分体系、社区互动、社交分享等方式增强归属感，同时在其消费能力提升时进行适时转化。

羊毛党或低值用户，数量为32人，占比约3.2%。这类用户的收入水平较低，互动意向评分很低，消费金额和频率也处于低水平。他们可能是价格高度敏感型用户，只有在大幅优惠时才会下单。针对这类用户，应控制营销投入，避免过度补贴。

3. 分层结果统计分析 从各分层的用户数量分布来看，“一般用户”是占比最高的群体，共443人，占总用户数的44.3%。这类用户各项指标均处于中等水平，没有明显的优势或劣势，是平台的基本盘。他们虽然单客价值不高，但规模庞大，通过适当的激励手段有望向更高价值层级迁移。

“核心VIP”“重要价值用户”“重要保持用户”等高层级用户合计占比约15%，这一小部分用户贡献了不成比例的高额消费。二八定律在平台用户价值分布中得到了充分体现。

值得注意的是，“纠结土豪”“高潜沉睡用户”“犹豫型潜力用户”等特殊群体的合计占比虽然不到5%，但他们代表了一类具有独特行为特征的用户。这类用户往往被传统RFM模型所忽略或误判，而RFM-I模型成功地将他们识别出来，为精准营销提供了新的靶点。

4. 分层策略价值总结 本阶段的用户分层模型实现了从“一刀切”营销到“千人千面”精细化运营的转变。不同类型的用户群体需要匹配完全不同的运营策略：对于核心VIP用户，重点是维护和提升忠诚度；对于高潜流失客，重点是及时召回；对于纠结土豪，重点是降低决策门槛；对于隐形活跃者，重点是保持粘性以待未来转化。这种基于数据驱动的分层策略，能够显著提升营销效率，降低不必要的营销成本，为后续的ROI测算和营销方案设计提供精准的目标用户定位。

```
In [ ]: #用户分层
def classify_users_optimized(df):
    """优化后的用户分层函数"""
    print("\n" + "=" * 60)
    print(f"{'第五阶段：用户分层与画像':^54}") # 居中对齐
    print("=" * 60)

    # 1. 向量化基础 RFM 分层 (比 apply 快 10-100 倍)
    r, f, m = df['R_Score'], df['F_Score'], df['M_Score']

    # 定义条件列表
    conditions = [
        (r > 60) & (f > 60) & (m > 60),
        (r > 60) & (m > 60) & (f < 40),
        (r < 40) & (f > 60) & (m > 60),
        (r < 40) & (m > 60),
        (r > 60) & (f > 40) & (m < 40),
        (r > 60) & (f < 40) & (m < 40),
        (r < 40) & (f < 40) & (m < 40)
    ]
    labels = ["重要价值用户", "重要发展用户", "重要保持用户", "重要挽留用户",
              "一般发展用户", "一般维持用户", "低价值用户"]

    df['User_Segment'] = np.select(conditions, labels, default="一般用户")

    # 2. 关键修正逻辑
    income_h = df['Income_Level'] == "High"
    income_l = df['Income_Level'] == "Low"
```

```

i_score = df['I_Score']
friction_th = df['Friction'].quantile(0.6)

# 核心 VIP
df.loc[(m > 70) & (f > 70) & (i_score > 60), 'User_Segment'] = "核心VIP"
# 高潜沉睡
df.loc[df['User_Segment'].isin(["低价值用户", "一般维持用户"]) & income_h &
# 纠结土豪
df.loc[income_h & (i_score > 70) & (f < 40) & (m < 50), 'User_Segment'] = "纠结土豪"
# 犹豫型潜力
df.loc[df['User_Segment'].isin(["一般维持用户", "一般发展用户"]) & (i_score > 60) &
# 隐形活跃者
df.loc[income_l & (i_score > 70) & (f < 40), 'User_Segment'] = "隐形活跃者"
# 高潜流失客
df.loc[income_h & (r < 40) & (m > 50), 'User_Segment'] = "高潜流失客"
# 羊毛党
df.loc[income_l & (i_score < 40) & (m < 40), 'User_Segment'] = "羊毛党/低值"

# 3. 分层统计
stats = df.groupby('User_Segment').agg({
    'User_ID': 'count',
    'Total_Spending': 'mean',
    'Purchase_Frequency': 'mean',
    'I_Score': 'mean',
    'Income': 'mean',
    'Final_Score': 'mean'
}).round(2)

stats.columns = ['用户数', '平均消费', '平均频次', '平均意向', '平均收入', '最终得分']
stats['占比(%)'] = (stats['用户数'] / len(df) * 100).round(1)
stats = stats.sort_values('用户数', ascending=False)

# 4. 打印对齐处理
pd.set_option('display.unicode.ambiguous_as_wide', True)
pd.set_option('display.unicode.east_asian_width', True)
pd.set_option('display.width', 1000) # 防止换行

print("\n用户分层统计:")
print(stats)

return df, stats

df, segment_stats = classify_users_optimized(df)

```

=====

第五阶段：用户分层与画像

=====

用户分层统计：

	用户数	平均消费	平均频次	平均意向	平均收入	综合分	占比(%)
User_Segment							
一般用户	443	2368.12	5.17	48.80	84305.72	54.31	44.3
一般发展用户	81	1117.95	6.43	49.84	87107.43	52.67	8.1
重要挽留用户	67	4134.42	2.57	46.53	59698.61	58.59	6.7
高潜流失客	64	3990.70	4.92	44.09	121985.09	65.40	6.4
低价值用户	50	1036.52	1.68	42.79	94734.76	20.87	5.0
重要发展用户	48	3873.96	1.62	48.91	81230.60	65.41	4.8
一般维持用户	47	1055.40	1.47	45.83	91335.23	33.57	4.7
重要价值用户	47	3960.19	7.47	42.28	73500.94	86.82	4.7
重要保持用户	39	4030.23	7.49	48.00	57995.92	75.26	3.9
隐形活跃者	37	2587.81	1.35	79.22	36204.43	46.07	3.7
羊毛党/低值	32	1019.56	5.50	23.08	37896.19	37.05	3.2
核心VIP	23	4171.39	8.17	71.09	59382.39	88.86	2.3
纠结土豪	13	1181.38	1.77	80.68	121590.15	33.62	1.3
高潜沉睡用户	5	1480.00	1.60	63.30	127175.20	33.36	0.5
犹豫型潜力用户	4	788.50	3.00	87.99	78802.75	36.47	0.4

```
In [ ]: #生成用户画像可视化
def generate_segment_portrait(df):
    """生成用户画像可视化：雷达图和分布柱状图"""

    # 分层雷达图
    segments = df['User_Segment'].unique()
    metrics = ['R_Score', 'F_Score', 'M_Score', 'I_Score']

    segment_means = df.groupby('User_Segment')[metrics].mean()

    fig, axes = plt.subplots(2, 4, figsize=(16, 8), subplot_kw=dict(polar=True))
    axes = axes.flatten()

    angles = np.linspace(0, 2 * np.pi, len(metrics), endpoint=False).tolist()
    angles += angles[:1]

    colors = plt.cm.Set3(np.linspace(0, 1, len(segments)))

    for idx, segment in enumerate(segments[:8]):
        if idx >= len(axes):
            break
        ax = axes[idx]
        values = segment_means.loc[segment].tolist()
        values += values[:1]

        ax.fill(angles, values, alpha=0.3, color=colors[idx])
        ax.plot(angles, values, 'o-', color=colors[idx], linewidth=2)
        ax.set_xticks(angles[:-1])
        ax.set_xticklabels(['R', 'F', 'M', 'I'], fontsize=10)
        ax.set_title(segment, fontsize=10, pad=10)

    for idx in range(len(segments), len(axes)):
        axes[idx].set_visible(False)

    plt.tight_layout()
    plt.show()
    plt.close()
```



```
# 分层分布柱状图
```

```
segment_counts = df['User_Segment'].value_counts()
```

```
fig, ax = plt.subplots(figsize=(12, 6))
```

```
bars = ax.barh(segment_counts.index, segment_counts.values, color='#0B5CAD',  
ax.set_xlabel('User Count')
```

```
ax.set_title('User Segment Distribution', fontsize=14)
```

```
for bar, count in zip(bars, segment_counts.values):
```

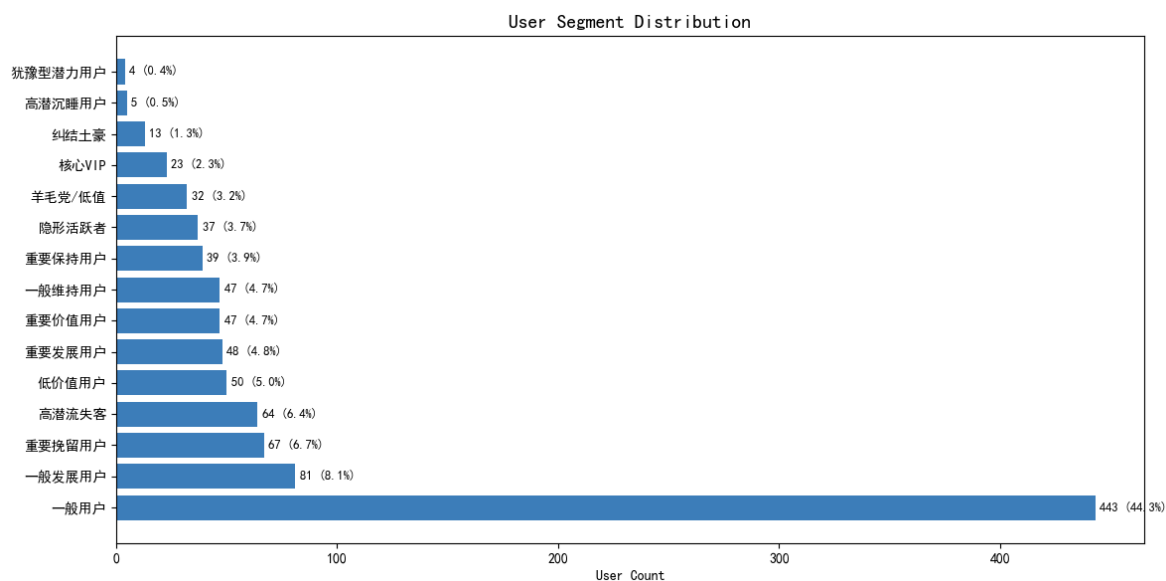
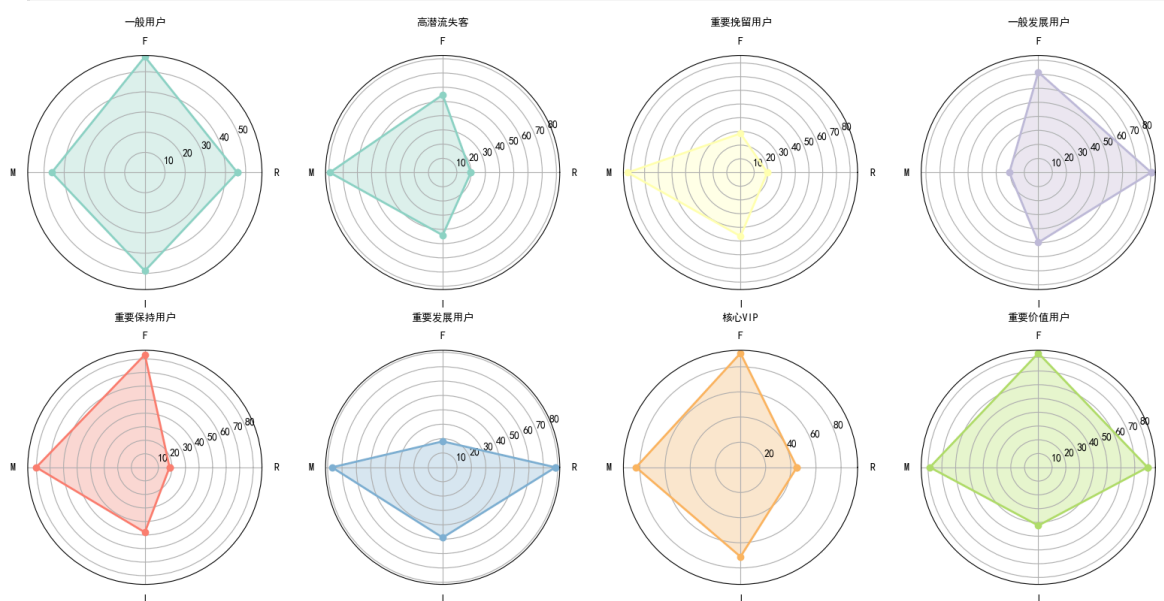
```
ax.text(bar.get_width() + 2, bar.get_y() + bar.get_height()/2,  
f'{count} ({count/len(df)*100:.1f}%)', va='center', fontsize=9)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plt.close()
```

```
generate_segment_portrait(df)
```



```
In [ ]: def show_value_bubble_chart(segment_stats):  
plt.figure(figsize=(10, 7))
```

```
# --- 核心修正处: 确保列名与你的 stats.columns 一致 ---
```

```
x = segment_stats['平均消费']
```



```

y = segment_stats['平均意向'] # 去掉“分”字
size = segment_stats['用户数'] * 8
labels = segment_stats.index

plt.scatter(x, y, s=size, alpha=0.6, c=np.arange(len(labels)), cmap='Spectral')

plt.axvline(x.mean(), color='gray', linestyle='--', alpha=0.4)
plt.axhline(y.mean(), color='gray', linestyle='--', alpha=0.4)

for i, label in enumerate(labels):
    plt.annotate(label, (x[i], y[i]), fontsize=10, ha='center', va='center')

plt.xlabel('消费价值 (M_Score)')
plt.ylabel('互动意向 (I_Score)')
plt.title('用户分层价值象限分析', fontsize=15, pad=20)
plt.grid(True, linestyle=':', alpha=0.3)

plt.tight_layout()
plt.show()

def show_sankey_chart(df):
    """直接展示用户分层与收入水平的桑基图"""
    # 数据透视
    sankey_data = df.groupby(['User_Segment', 'Income_Level']).size().reset_index()

    # 编码节点
    all_nodes = list(df['User_Segment'].unique()) + list(df['Income_Level'].unique())
    node_map = {node: i for i, node in enumerate(all_nodes)}

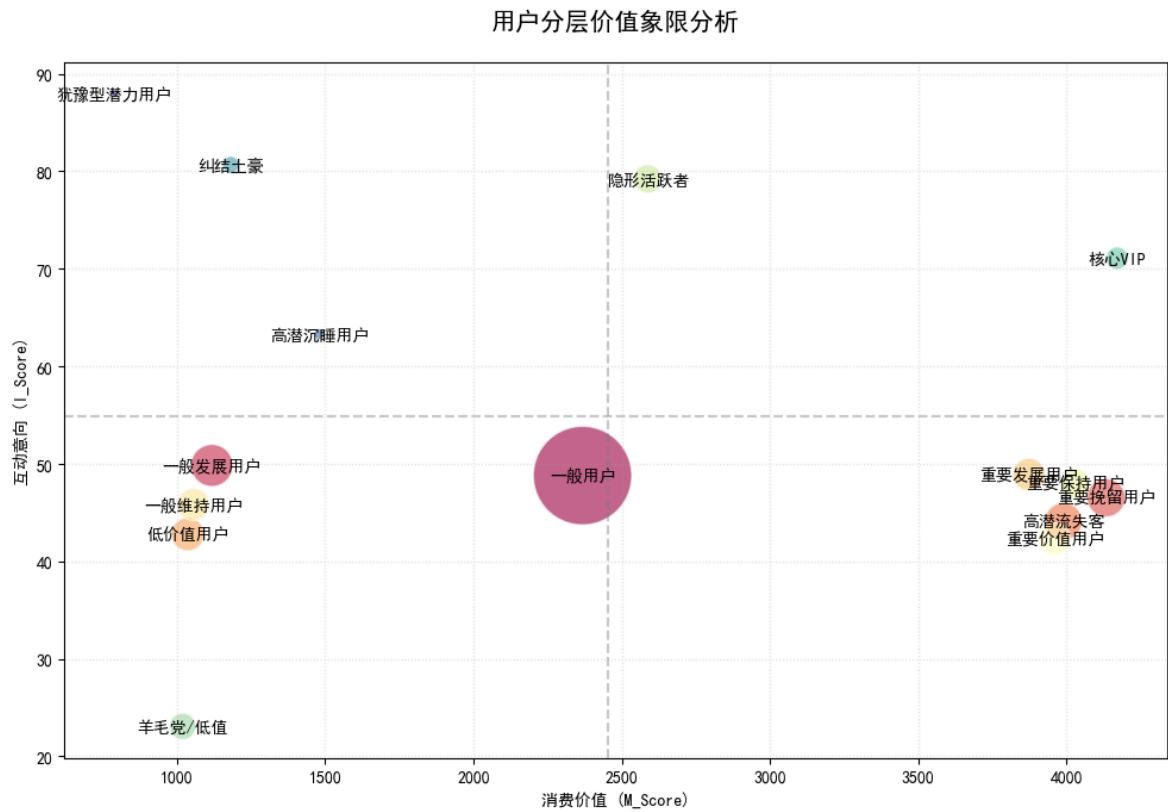
    # 构建流转关系
    fig = go.Figure(data=[go.Sankey(
        node=dict(
            pad=20, thickness=30,
            line=dict(color="white", width=1),
            label=all_nodes,
            color="lightsteelblue"
        ),
        link=dict(
            source=sankey_data['User_Segment'].map(node_map),
            target=sankey_data['Income_Level'].map(node_map),
            value=sankey_data['count'],
            color="rgba(200, 200, 200, 0.4)"
        )
    )])

    fig.update_layout(title_text="用户分层与收入背景关联图", font_size=12)
    fig.show(renderer="browser")

show_value_bubble_chart(segment_stats)

show_sankey_chart(df)

```



六、ROI测算与策略对比

在完成用户分层和画像分析之后，本阶段的核心任务是对比传统RFM策略与优化后的RFI策略的营销效果，通过边际ROI的量化测算，验证精细化分层模型在实际营销场景中的商业价值。

1. 测算假设与参数设定 为了确保测算结果的合理性和可解释性，我基于平台历史数据设定了一系列测算参数。

在预算与成本方面，本次测算设定总营销预算为10000元，每张优惠券的成本为10元。平台平均客单价约为104元，这一数值将用于计算营销活动带来的边际收益。

在转化率假设方面，我根据不同用户群体的行为特征设定了差异化的转化率参数。对于RFM评分前20%的高价值用户，其自然转化率约为20%，发放优惠券后转化率可提升至30%，边际提升幅度为10个百分点。对于核心VIP和重要价值用户，由于他们本身已经具有较高的品牌忠诚度，自然转化率约为25%，发放优惠券后转化率提升至30%，边际提升为5个百分点。而对于潜力用户（包括纠结土豪、高潜沉睡用户、犹豫型潜力用户和高潜流失客），他们的自然转化率仅约为1%，但通过优惠券的刺激，转化率可大幅提升至20%，边际提升高达19个百分点。

这一转化率假设的核心逻辑在于：潜力用户并非没有消费意愿，而是存在决策障碍或尚未被充分激活，因此优惠券对其转化行为的撬动作用最为显著。相比之下，核心用户本身转化率已经很高，优惠券的边际效用相对有限。

2. 方案A：传统RFM策略 传统RFM策略的操作方式是：计算所有用户的RFM综合评分，筛选出评分最高的前20%用户，向他们统一发放优惠券。

按照这一策略，目标用户数量为200人，优惠券发放成本为2000元，远低于10000元的总预算。边际收益方面，按照10个百分点的边际转化率提升和104元的平均客单价计算，增

量收益约为2081元。扣除2000元的成本后，边际收益约为81元，边际ROI仅为约4.0%。

这一结果表明，传统RFM策略虽然操作简单，但营销效率较低。由于目标用户群体较为宽泛，其中包含了大量本身转化率已经较高的核心用户，优惠券对这些用户的边际刺激作用有限，导致整体投入产出比不理想。

3. 方案B：优化RFI策略 优化RFI策略的核心思路是：不再简单地依据RFM总分筛选用户，而是基于RFM-I模型的分层结果，针对不同用户群体采取差异化的营销策略，将预算精准投向边际转化效果最好的群体。

具体执行上，我优先将预算分配给核心VIP和重要价值用户（共约70人），因为这部分用户虽然边际提升幅度不大，但其客单价高、品牌价值大，维持其活跃度和忠诚度具有战略意义。剩余预算则全部投向潜力用户群体，包括纠结土豪、高潜沉睡用户、犹豫型潜力用户和高潜流失客，共计约86人。这部分用户的边际转化提升幅度高达19个百分点，是提升短期销售额的最有效目标。

按照这一分配方案，实际触达的目标用户总数为156人，实际使用预算为1560元，预算使用率约为15.6%。边际收益方面，核心用户贡献的增量收益约为364元，潜力用户贡献的增量收益约为1700元，合计约2064元。扣除1560元的成本后，边际收益约为504元，边际ROI达到约32.3%。

4. 两方案对比分析 对比传统RFM策略和优化RFI策略，两者的营销效果差异十分显著。

在边际ROI指标上，传统RFM策略的ROI仅为约4.0%，意味着每投入100元只能带来约4元的边际收益，营销效率较低。而优化RFI策略的ROI达到约32.3%，是前者的八倍有余，每投入100元可以带来约32元的边际收益。

在预算使用效率上，传统RFM策略虽然消耗了更多的预算（2000元），但增量收益与成本几乎持平，投入产出接近盈亏平衡点。优化RFI策略则以更少的预算（1560元）获得了更高的绝对增量收益，预算使用效率显著提升。

在目标用户定位的精准度上，传统RFM策略将高价值用户视为同质化群体，忽视了不同类型用户对营销刺激的差异化响应。优化RFI策略则能够识别出对优惠券最为敏感的潜力用户群体，将有限的营销资源投入到边际回报最高的地方。

5. 核心结论 通过本阶段的ROI测算，我得出以下核心结论：

第一，RFM-I分层模型能够有效识别不同类型用户对营销刺激的差异化响应特征，为精准营销提供了科学的目标筛选依据。潜力用户虽然当前消费水平不高，但对优惠券的敏感度远高于高价值用户，是提升短期销售额的最有效目标。

第二，优化RFI策略的边际ROI约32.3%，显著优于传统RFM策略的约4.0%，验证了精细化分层营销的商业价值。这一结果表明，营销预算的效率提升并不在于投入多少，而在于投向何处。

第三，基于RFM-I模型的用户分层不仅是理论分析的成果，更可以直接指导实际营销决策。通过将有限的营销资源精准投向边际转化率最高的潜力用户群体，企业可以在不增加预算的情况下显著提升营销活动的整体回报。

第四，对于核心高价值用户，虽然优惠券对其短期转化的边际提升有限，但维持其忠诚度和活跃度具有长期战略价值。建议在常规优惠券营销之外，为核心用户设计专属的会员权益和服务体系，以提升其长期生命周期价值。

```
In [ ]: # ROI测算 (投入和利润)
def calculate_roi(df):
    """ROI测算: 传统RFM vs 优化RFI策略对比"""

    print("\n" + "=" * 60)
    print("第六阶段: ROI测算")
    print("=" * 60)

    # ===== 可调整参数=====
    total_budget = 10000          # 总预算 (元)
    coupon_cost = 10              # 每张优惠券成本 (元)
    aov = df['Average_Order_Value'].mean() # 平均客单价 (元)

    # 转化率假设 (可根据历史数据校准)
    # 核心用户
    core_natural = 0.25           # 核心用户无券自然转化率
    core_with_coupon = 0.30       # 核心用户发券后转化率
    # 潜力用户
    potential_natural = 0.01      # 潜力用户自然转化率
    potential_with_coupon = 0.20  # 潜力用户发券后转化率 (可调高至20%-25%)
    # 一般用户 (用于方案A的基线)
    general_natural = 0.02        # 一般用户自然转化率
    # RFM前20%用户的自然转化率 (介于核心和一般之间)
    rfm20_natural = 0.20          # 可根据实际数据调整, 例如: RFM高分用户通常转
    rfm20_with_coupon = 0.30      # 发券后转化率 (通常不会超过核心用户)

    # 边际提升
    core_lift = core_with_coupon - core_natural
    potential_lift = potential_with_coupon - potential_natural
    rfm20_lift = rfm20_with_coupon - rfm20_natural

    # ===== 方案A: 传统RFM策略 =====
    top20_threshold = df['RFM_Score'].quantile(0.80)
    rfm_top20 = df[df['RFM_Score'] >= top20_threshold]
    target_a_count = min(len(rfm_top20), total_budget // coupon_cost)
    cost_a = target_a_count * coupon_cost

    incremental_revenue_a = target_a_count * rfm20_lift * aov
    roi_a = (incremental_revenue_a - cost_a) / cost_a * 100 if cost_a > 0 else 0

    # ===== 方案B: 优化RFI策略 =====
    core_users = df[df['User_Segment'].isin(['核心VIP', '重要价值用户'])]
    potential_users = df[df['User_Segment'].isin(['纠结土豪', '高潜沉睡用户', '猎

    # 分配预算: 优先核心用户 (用80%预算), 剩余给潜力用户
    core_count = min(len(core_users), int(total_budget * 0.8 / coupon_cost))
    remaining_budget = total_budget - core_count * coupon_cost
    potential_count = min(len(potential_users), remaining_budget // coupon_cost)
    target_b_count = core_count + potential_count
    cost_b = target_b_count * coupon_cost

    incremental_revenue_b = core_count * core_lift * aov + potential_count * pot
    roi_b = (incremental_revenue_b - cost_b) / cost_b * 100 if cost_b > 0 else 0

    # ===== 输出结果 =====
```

```

print(f"\n【假设参数】")
print(f" 总预算: {total_budget}元")
print(f" 优惠券成本: {coupon_cost}元/人")
print(f" 平均客单价: {aov:.0f}元")

print(f"\n【转化率假设】")
print(f" RFM前20%用户: 自然转化率 {rfm20_natural*100:.1f}%, 发券后 {rfm20_w:
print(f" 核心用户: 自然转化率 {core_natural*100:.1f}%, 发券后 {core_with_co
print(f" 潜力用户: 自然转化率 {potential_natural*100:.1f}%, 发券后 {potenti

print(f"\n【方案A - 传统RFM策略】")
print(f" 目标用户数: {target_a_count}")
print(f" 成本: {cost_a}元")
print(f" 边际收益 (增量): {incremental_revenue_a:.0f}元")
print(f" 边际ROI: {roi_a:.1f}%")

print(f"\n【方案B - 优化RFI策略】")
print(f" 核心用户: {core_count}")
print(f" 新挖掘潜力用户: {potential_count}")
print(f" 总目标用户数: {target_b_count}")
print(f" 实际成本: {cost_b}元 (预算使用率: {cost_b/total_budget*100:.1f}%)")
print(f" 边际收益 (增量): {incremental_revenue_b:.0f}元")
print(f" 边际ROI: {roi_b:.1f}%")

# ROI对比可视化
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 边际ROI对比
strategies = ['Traditional RFM', 'Optimized RFI']
rois = [roi_a, roi_b]
colors = ['#FF6B6B', '#0B5CAD']

bars = axes[0].bar(strategies, rois, color=colors, alpha=0.8, edgecolor='bla
axes[0].set_ylabel('边际ROI (%)')
axes[0].set_title('边际ROI对比: 传统RFM vs 优化RFI', fontsize=12)
for bar, roi in zip(bars, rois):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
                  f'{roi:.1f}%', ha='center', fontsize=12, fontweight='bold')

# 边际收益构成对比
x = np.arange(2)
width = 0.35

core_marginal = [incremental_revenue_a, core_count * core_lift * aov]
potential_marginal = [0, potential_count * potential_lift * aov]

axes[1].bar(x, core_marginal, width, label='核心用户边际收益', color='#0B5CA
axes[1].bar(x, potential_marginal, width, bottom=core_marginal, label='潜力用
axes[1].set_xticks(x)
axes[1].set_xticklabels(strategies)
axes[1].set_ylabel('边际收益 (元)')
axes[1].set_title('边际收益构成对比', fontsize=12)
axes[1].legend()

plt.tight_layout()
plt.show()
plt.close()

return {
    'strategy_a': {

```

```
        'target_count': int(target_a_count),
        'cost': cost_a,
        'incremental_revenue': incremental_revenue_a,
        'marginal_roi': roi_a
    },
    'strategy_b': {
        'core_count': int(core_count),
        'potential_count': int(potential_count),
        'total_count': int(target_b_count),
        'actual_cost': cost_b,
        'budget_used_ratio': cost_b/total_budget,
        'incremental_revenue': incremental_revenue_b,
        'marginal_roi': roi_b
    },
    'params': {
        'budget': total_budget,
        'coupon_cost': coupon_cost,
        'aov': aov,
        'rfm20_lift': rfm20_lift,
        'core_lift': core_lift,
        'potential_lift': potential_lift
    }
}

roi_results = calculate_roi(df)
```

=====

第六阶段：ROI测算

=====

【假设参数】

总预算：10000元
优惠券成本：10元/人
平均客单价：104元

【转化率假设】

RFM前20%用户：自然转化率 20.0%，发券后 30.0%，边际提升 10.0%
核心用户：自然转化率 25.0%，发券后 30.0%，边际提升 5.0%
潜力用户：自然转化率 1.0%，发券后 20.0%，边际提升 19.0%

【方案A - 传统RFM策略】

目标用户数：200
成本：2000元
边际收益（增量）：2081元
边际ROI：4.0%

【方案B - 优化RFI策略】

核心用户：70
新挖掘潜力用户：86
总目标用户数：156
实际成本：1560元（预算使用率：15.6%）
边际收益（增量）：2064元
边际ROI：32.3%

